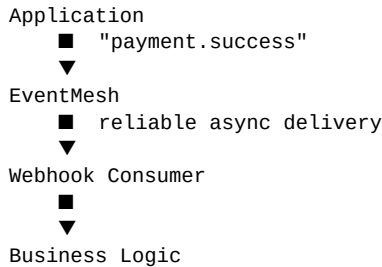


EventMesh — Backend Foundation Notes

Steps 1–5 • Quick Revision + Complete Process Flow

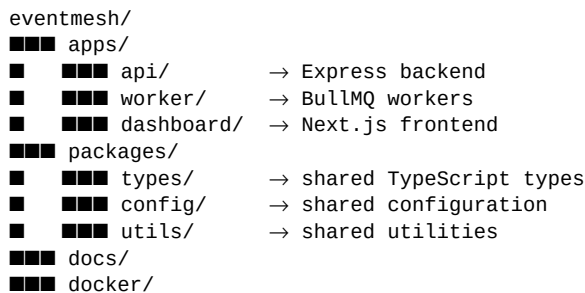
1. EventMesh kya hai?

EventMesh = Reliable Event Delivery Infrastructure. Iska kaam application ke events ko reliably registered consumers/webhooks tak deliver karna hai.



EventMesh khud payment, email/SMS ya business logic nahi karta. Ye infrastructure concerns handle karega: event ingestion, authentication, authorization, queueing, webhook delivery, retries, exponential backoff, timeout, delivery tracking, DLQ, HMAC signatures, rate limiting, idempotency aur multi-tenancy.

2. Monorepo kyun?

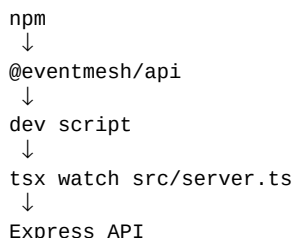


Sab applications ek Git repository mein hain, isliye ise monorepo kehte hain. Root package.json mein npm workspaces apps/* aur packages/* ko workspaces ke roop mein identify karta hai.

3. --workspace ka meaning

```
npm run dev --workspace=@eventmesh/api
```

Iska matlab hai: **@eventmesh/api** workspace ka **dev** script run karo. API ke package.json mein dev script **tsx watch src/server.ts** hai.



--workspace = specific workspace. **--workspaces** = all workspaces.

4. Current technology stack

Node.js	→ backend runtime
TypeScript	→ type safety
Express.js	→ HTTP server + routing
cors	→ cross-origin requests
helmet	→ security-related HTTP headers

dotenv → environment variables
tsx → development mein TypeScript run karna

Later PostgreSQL, Prisma, Redis, BullMQ, Next.js, Docker aur k6 add honge.

5. server.ts vs app.ts

server.ts ka main kaam actual HTTP server start karna hai. **app.ts** Express application ko configure karta hai.

```
server.ts
  ↓
app import
  ↓
app.listen(...)
  ↓
Port 3000

app.ts
  ↓
middleware
  ↓
routes
  ↓
error handler
```

Ye separation testing aur production architecture ke liye useful hai.

6. Current API structure

```
apps/api/
  ■■■ src/
  ■   ■■■ config/
  ■   ■   ■■■ env.ts
  ■   ■■■ controllers/
  ■   ■   ■■■ health.controller.ts
  ■   ■■■ middleware/
  ■   ■   ■■■ error.middleware.ts
  ■   ■   ■■■ request-logger.middleware.ts
  ■   ■■■ routes/
  ■   ■   ■■■ health.routes.ts
  ■   ■■■ services/
  ■   ■■■ utils/
  ■   ■   ■■■ app-error.ts
  ■   ■■■ app.ts
  ■   ■■■ server.ts
  ■■■ .env
  ■■■ .env.example
  ■■■ package.json
  ■■■ tsconfig.json
```

7. Middleware

Middleware request aur final handler ke beech execute hota hai. `next()` request ko next layer mein bhejta hai.

```
Request
  ↓
Middleware 1
  ↓
Middleware 2
  ↓
Route
  ↓
Controller
  ↓
Response
```

Current middleware: Helmet, CORS, `express.json()` aur request logger.

8. Routes vs Controllers

Route decide karta hai ki kis HTTP request ko kis handler ke paas bhejna hai. **Controller** HTTP request/response handle karta hai.

```
GET /health
  ↓
health.routes.ts
  ↓
healthCheck()
  ↓
JSON response
```

9. /health request ka complete flow

```
Browser
  ↓
GET http://localhost:3000/health
  ↓
Node.js
  ↓
Express
  ↓
Helmet
  ↓
CORS
  ↓
JSON parser
  ↓
Request logger
  ↓
Health Router
  ↓
Health Controller
  ↓
HTTP 200
  ↓
JSON Response

{
  "status": "ok",
  "service": "eventmesh-api"
}
```

10. Centralized Error Handling

Production backend mein invalid input, database failure, Redis failure, timeout aur unexpected exceptions aa sakte hain. Har controller mein manually error response likhne ke bajay centralized error handler use karte hain.

```
Controller
  ↓
throw error
  ↓
Central Error Middleware
  ↓
Consistent HTTP response
```

Custom `AppError` mein message, statusCode aur machine-readable code hota hai. Example: `EVENT_NOT_FOUND` + HTTP 404.

11. Request Logging

Request logger HTTP method, URL, status code aur response time record karta hai.

```
GET /health 200 - 5ms
```

Later is basic logger ko structured production logging aur request IDs ke saath improve kiya ja sakta hai.

12. Configuration

Environment variables ko central config/env.ts se manage karna hai. Typical variables: PORT, DATABASE_URL, REDIS_URL, HMAC_SECRET.

.env mein local/real values hoti hain aur .env.example mein required variable names. Real secrets ko GitHub par commit nahi karna.

13. TypeScript development vs production

```
Development:
TypeScript → tsx → Node.js

Production:
TypeScript → tsc → dist/*.js → Node.js
```

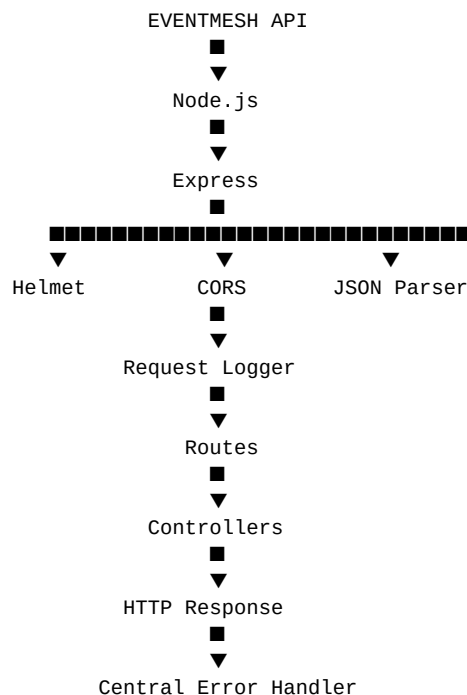
Root tsconfig common configuration deta hai; apps/api/tsconfig.json use extend karta hai. strict: true type safety enforce karta hai.

14. Git strategy

```
chore: initialize monorepo
↓
feat: add express api server
↓
refactor: structure express api layers
↓
feat: add error handling and request logging
```

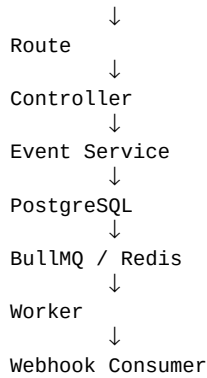
Har commit ek meaningful milestone represent karega. Isse GitHub par genuine development history dikhegi.

15. Current EventMesh backend mental model



16. Future complete request flow

```
POST /v1/events
↓
Express
↓
Authentication
↓
Rate Limiting
↓
Validation
```



17. 1-Month Revision — One-line mental model

Server request accept karta hai → middleware request process karta hai → route request ko controller tak bhejta hai → controller service/application logic trigger karta hai → data layer database se interact karti hai → response client ko return hota hai → errors centralized handler se manage hote hain.

18. Current milestone

Steps 1–5 complete: monorepo foundation, Express + TypeScript API, health endpoint, layered structure, centralized error handling aur request logging. Next major milestone: PostgreSQL + Prisma foundation.